# Four Structural Weaknesses of the GDTW Similarity Metric

Diagnosis, Empirical Evidence, and Proposed Remedies

Yuli Tshuva

August 2026

**Abstract**

Four known weaknesses of the segment-based similarity metric ($GDTW$): (i) the change-point detector is a heuristic stack with no stability guarantee; (ii) the merge rule is governed by an unmotivated constant; (iii) the segment representation misrepresents scale on both axes; (iv) the paradigm collects human judgements but the model contains no learning. Each is diagnosed against the implementation, measured on the project corpus (10,000 mined stock-price sequences), and matched to a remedy from the literature. Three further defects found during the audit are in §6; §7 ranks all recommendations. **Reading order for the author:** §7 (the whole report as a ranked list), then §6, then §5.

## Contents



## 1 The method under review

GDTW (`seq_sim_alg.py:seq_distance`) min–max normalises each sequence (`normalize_sequence`); segments it at change points (`change_points_detection`); maps each segment to a 12-dimensional feature vector $\phi(s)$ (`extract_segment_features`), standardised by hard-coded constants and scaled componentwise by weights $w \in \Delta^{11}$; then aligns the two segment chains with a merge-aware

dynamic program (`dtw_merge`). A merge step consuming $d_i$ segments of $f$ and $d_j$ of $g$ costs

$$\begin{aligned} \text{cost} &= \frac{L_x + L_y}{2}\left(\Delta_{ij} + P_{ij}\right), \qquad \Delta_{ij} = \underbrace{\left\|\mu(X_{i-d_i:i}) - \mu(Y_{j-d_j:j})\right\|_2}_{\text{merged feature distance}}, \\ P_{ij} &= \underbrace{(\lambda_1 - 3\lambda_2\rho_x)(d_i - 1) \;+\; (\lambda_1 - 3\lambda_2\rho_y)(d_j - 1)}_{\text{merge penalty}}, \end{aligned} \tag{1}$$

with $L_x, L_y$ the relative durations, $\mu$ the feature-merge operator (`merge_sequence`), $\rho$ a correlation over trend features (`features_correlation`), and $\lambda_1, \lambda_2$ the mean and s.d. of the segment-to-segment distance matrix. All measurements below use `data/six_cps_sequences.npz` (10,000 sequences, length 1000, mean 5.93 change points); scripts are in `reports/diagnostics/`.

## 2 Weakness 1: the change-point detector

`change_points_detection` chains five heuristics, each with its own constant: a ternary sign map of $f'$ at $\tau = 0.10 \cdot \text{der\_amp}$ where $\text{der\_amp} = \frac{1}{2}(\max f' - \min f')$, summed with a second map at $\tau/3$ (`utils.py:204--205`); `feature_points` (emits a point pair when the running sign window exceeds 2% of length); `find_local_extrema` (smoothed-derivative zero crossings, 5% prominence filter); `drop_false_extrema`; and `merge_nearby_points` (removes points closer than $\max(5, 1\%)$, then midpoint-merges points within 1% in height).

**(a) No objective.** Nothing is optimised. There is no cost function whose minimiser is the returned segmentation, hence no notion of correctness, no way to compare two segmentations, and no fit-versus-complexity trade-off. Each stage greedily edits the previous one's output; early errors are unrecoverable.

**(b) The governing threshold is maximally non-robust.** $\tau \propto \max f' - \min f'$ — the derivative *range*, fixed by 2 samples out of 999, with unbounded sensitivity to one outlier. It governs what counts as "changing" everywhere, so one noisy sample rescales the whole segmentation. The same statistic recurs in `extract_segment_features` (`utils.py:316`), so the *descriptors* inherit the fragility.

**(c) Six unidentified constants.** 2% (segment percentage), 10% (change threshold), the $\tau/3$ split, 5% (prominence), 1% (minimum distance), 1% (height merge). None fitted, none with a sensitivity analysis.

**(d) One third of the detector is dead code.** `find_local_extrema` (`utils.py:213`) feeds a list `combined` (`utils.py:217--232`) that is never read — line 234 proceeds from `signs_fps`. Verified: stubbing `find_local_extrema` to return `[]` leaves segmentation **bit-identical on 300/300 sequences**. About 40 lines, including all prominence logic, affect no published result.

### 2.1 Evidence

60 random sequences, perturbed and re-segmented. Hausdorff = boundary displacement as % of sequence length.

Table 1: Segmentation stability under perturbation (mean 5.93 change points).

| Perturbation | % seq. with changed $K$ | mean $\lvert\Delta K\rvert$ | median Hausdorff | p90 Hausdorff |
|---|---|---|---|---|
| Gaussian noise $\sigma = 0.005$ | **100.0%** | **3.92** | **41.8%** | 48.6% |
| Gaussian noise $\sigma = 0.01$ | 100.0% | 3.92 | 41.8% | 48.6% |
| Time resample $\times 1.05$ | 0.0% | 0.00 | 0.1% | 0.3% |
| Time shift, 1% of length | 11.7% | 0.12 | 1.0% | 2.2% |
| Amplitude rescale $\times 0.98$ | 0.0% | 0.00 | 0.0% | 0.0% |
| *PELT (`ruptures`, $\ell_2$), noise $\sigma = 0.01$* | 32.0% | 0.40 | 1.5% | — |

A 0.5%-of-range perturbation — invisible at plotting resolution — changes the segment count in *every* sequence, by two thirds of the mean count, and displaces boundaries by 42% of the sequence length. That is a different segmentation, not a perturbed one. PELT under twice the noise is 10× more stable in count, 28× in localisation.

**Mechanism.** Diagnosis (b) confirmed: under $\sigma = 0.005$, der_amp inflates by a median factor of **6.7** (p90: 10.9), $\tau$ with it, and the fraction of samples deemed "not changing" rises from 13.3% to 26.0% — the detector goes half blind. A robust scale (derivative IQR) inflates only 4.15×.

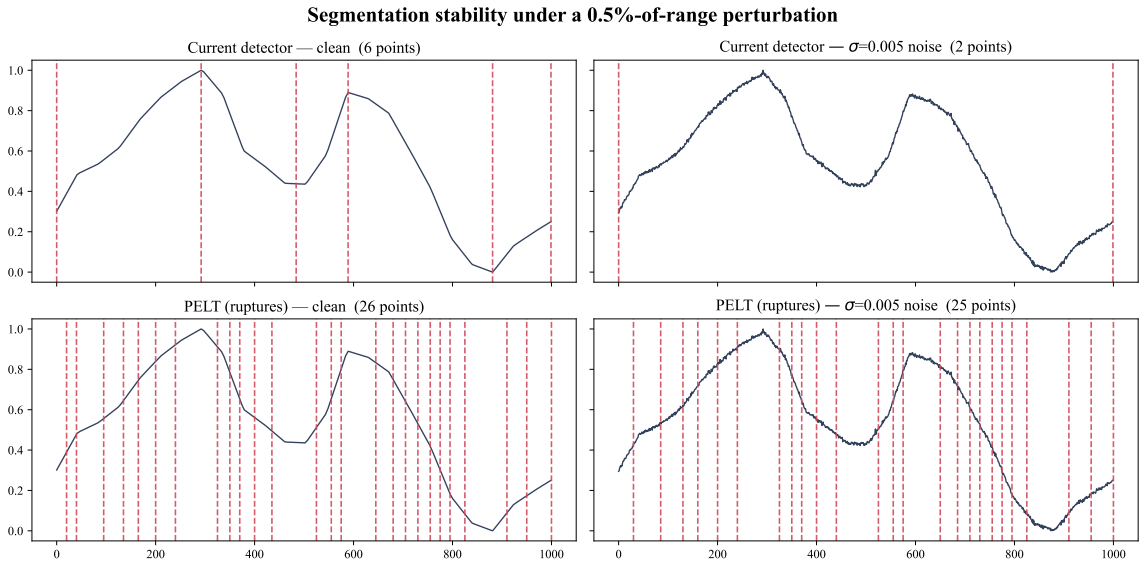


Figure 1: Top: current detector, clean and perturbed — six change points collapse to two. Bottom: PELT on the same signals, 26 → 25. PELT's penalty ($\beta = 0.05$) was not tuned for parsimony, so this compares *stability*, not granularity.

## 2.2 Remedy

The field formulates segmentation as *penalised model selection*: a cost function, a search strategy, a constraint on the number of changes [3]. The `ruptures` package is already in `requirements.txt` but unused.

1. **Replace the stack with $\ell_1$ trend filtering [6]** — $\min_x \frac{1}{2}\|y-x\|_2^2+\lambda\|D^{(2)}x\|_1$, convex, solution exactly piecewise linear with knots at the kinks. Best fit to GDTW's needs: it denoises *and* segments in one step, returns the per-segment trend model that §4 needs, has one parameter with

a known regularisation path, and being convex is unique and stable. Alternative: **PELT [4]** with a piecewise-linear cost (`CostLinear`, or $\ell_2$ on the derivative), exact in expected linear time, one parameter $\beta$ settable by BIC/MDL or learned (§5). For outlier robustness see Fearnhead & Rigaill [5], the principled version of what (b) gets wrong. FLUSS/FLOSS [8] is domain-agnostic but detects *regime* change via motif recurrence — a different notion from visual gestures; not recommended here.

2. **Keep PIP [1] as comparator** — recursive maximal-deviation-from-chord selection, deterministic, one parameter, yields a *nested hierarchy*, and was designed for human-perceived shape in financial charts. If conclusions survive swapping the segmenter, the contribution is the alignment, which strengthens the paper. Its hierarchy also turns §4's merge decision into a choice of *level*.
3. **Delete or wire in the dead branch**, then re-run every published result.
4. **Report stability as a metric**: median Hausdorff and $|\Delta K|$ at $\sigma \in \{0.005, 0.01, 0.02\}$. A metric claiming to model human perception cannot have a representation that moves under noise humans cannot see.

## 3 Weakness 2: misrepresentation of scale

**(a) Features are dimensionally inhomogeneous in time.** With $\Delta t$ the sampling interval: `mean_curvature` $= \overline{|\Delta^2 s|}$ scales as $\mathcal{O}(\Delta t^2)$; `mean_diff` and `mean_abs_diff` as $\mathcal{O}(\Delta t)$; `sum_abs_diff`, `amplitude`, `mean_value`, `length` as $\mathcal{O}(1)$. So resampling a segment without touching its drawn shape changes its descriptor. Resampling a real segment (192 samples, sequence 3050) by $\{0.5, 1, 2, 4\}$, rescaling its container identically so the rendering is unchanged:

Table 2: One segment at four sampling resolutions; the shape is identical throughout.

| Feature | ×0.5 | ×1 | ×2 | ×4 | max/min |
|---|---|---|---|---|---|
| `mean_curvature` | 0.000260 | 0.000064 | 0.000016 | 0.000004 | **65.7** |
| `mean_diff` | −0.00592 | −0.00294 | −0.00147 | −0.00073 | **8.07** |
| `mean_abs_diff` | 0.00592 | 0.00294 | 0.00147 | 0.00073 | **8.07** |
| `amplitude` *(and the 8 others)* | 0.56232 | 0.56232 | 0.56232 | 0.56232 | 1.00 |

Observed ratios match the predicted exponents exactly ($8^1 = 8.07$, $8^2 = 64 \approx 65.7$). This is masked only because `mine_data.py` interpolates everything to 1000 points: **the pipeline is self-consistent solely because of a preprocessing step a real deployment would not have**. Any variable-length or multi-resolution use breaks it silently, and sampling-rate invariance — required of a perceptual metric — cannot be claimed.

**(b) Global normalisation makes segment scale context-dependent.** Because `normalize_sequence` min–max scales the *whole* sequence, a segment's `amplitude` and `mean_value` measure it against extremes set by parts of the sequence it has nothing to do with. The same 200-sample motif placed before four different tails:

Table 3: One motif, four contexts.

| Tail slope | 0.1 | 0.5 | 2.0 | 5.0 |
|---|---|---|---|---|
| `amplitude` of the motif | 1.0000 | 0.4615 | 0.1395 | **0.0583** |
| `mean_value` of the motif | 0.5000 | 0.2308 | 0.0698 | 0.0291 |

A 17× swing with the motif untouched (Fig. 2). Min–max is also non-robust: one outlier compresses everything else. The trend fractions are unaffected *here* only because the tail is monotone; in general they move too, since their threshold derives from the global der_amp.

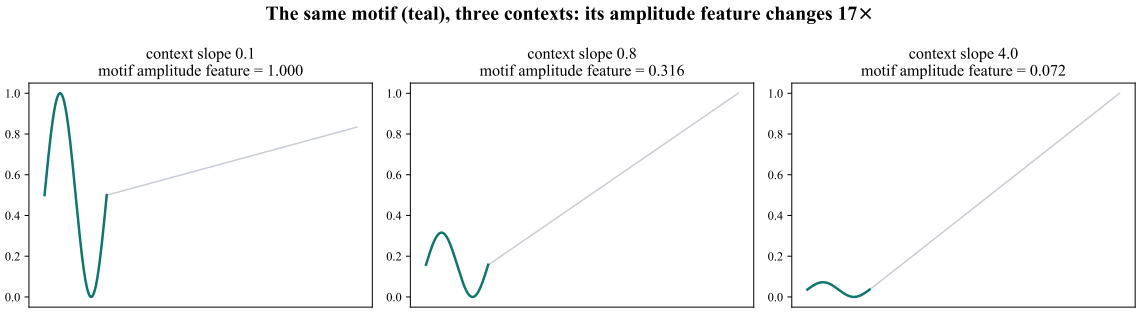


Figure 2: The identical motif (teal) in three contexts. Its `amplitude` feature — which the metric treats as a property of the motif — is a property of the tail.

**(c) Duration is double-counted, and the features are collinear.** In Eq. 1 relative duration enters *multiplicatively* as $(L_x + L_y)/2$ *and* additively as `length` inside the norm, with no stated semantics for the composite. Over 1489 real segments:

| Pair | $r$ | Pair | $r$ |
|---|---|---|---|
| `sum_abs_diff` ∼ `amplitude` | +0.997 | `mean_diff` ∼ `sharp_increasing` | +0.844 |
| `sum_abs_diff` ∼ `length` | +0.847 | `mean_diff` ∼ `sharp_decreasing` | −0.838 |
| `amplitude` ∼ `length` | +0.845 | \|`mean_diff` × `length`\| ∼ `amplitude` | +1.000 |

Segments are near-monotone by construction, so total variation *is* amplitude ($r = 0.997$) and slope × duration *is* amplitude exactly ($r = 1.000$). The twelve "independent perceptual axes" carry 95% of their variance in **7 components**, with condition number $4 \times 10^{12}$. Consequence for §5: a diagonal weight vector over collinear features is **not identifiable**.

**(d) No aspect ratio.** Humans judge a *rendering*, and perceived slope depends on aspect ratio — "banking to 45°" [18], refined empirically by [19]; Gogolou et al. [20] show similarity perception in time series depends on the visual encoding. GDTW has no parameter for the horizontal/vertical trade-off; it is implicit across twelve weights, hence neither inspectable nor learnable.

## 3.1 Remedy

1. **Explicit anisotropy $\rho$.** Represent a segment spanning $(\Delta t, \Delta y)$ by angle $\theta = \arctan(\rho\,\Delta y/\Delta t)$ and arclength $\ell = \sqrt{(\Delta t)^2 + (\rho\,\Delta y)^2}$. One scalar carries the whole $x$-vs-$y$ trade-off, is interpretable (it *is* the aspect ratio the annotators saw), and is learnable — making the perceptual claim testable rather than implicit.

2. **Make every feature dimensionless.** $\theta$ for `mean_diff`; scale-free curvature (curvature $\times$ arclength, or integrated turning angle) for `mean_curvature`; amplitude as a fraction of a robust global scale; length as a fraction of $T$. Verify by re-running Table 2: every ratio must be 1.00.
3. **Remove redundancy.** Drop `sum_abs_diff` outright ($r = 0.997$ with `amplitude`: it adds only an unidentifiable parameter). Keep two of {slope, amplitude, duration}, or keep all and replace diagonal weights with a learned low-rank PSD matrix (§5), which handles collinearity instead of double-counting it.
4. **Robustify normalisation.** Percentile range ($[p_1, p_{99}]$) instead of min–max, plus a *local* amplitude feature (segment vs. neighbours) alongside the global one, so the representation records both "how big in the chart" and "how big relative to its surroundings".
5. **Fix the double count.** Decide whether duration weights the cost or is a feature, and say which. Weighting the cost is standard (it makes the distance an integral over the sequence); then `length` leaves the feature vector.

# 4 Weakness 3: the merge rule and its magic constant

The penalty is $(\lambda_1 - 3\lambda_2\rho)(d-1)$.

**(a) The constant 3 has no derivation.** It converts a standard deviation into a penalty unit, determines when merging is worthwhile, and has no sensitivity analysis.

**(b) The correlation term measures noise.** `features_correlation` takes the $d \times 5$ trend block, computes Pearson $r$ *between rows* — across five values, where the standard error is near 0.5 — and returns the minimum over pairs. Over 278 merge candidates from 40 sequence pairs, $\rho$ is **negative in 87.4% of cases** (mean $-0.497$, min $-0.980$). It was meant to *reduce* the penalty for segments that belong together; it almost always inflates it.

**(c) The minimum over pairs breaks the intended linearity.** $\rho = \min$ is non-increasing in $d$, so the penalty grows faster than $(d-1)$ by an amount set by the number of pairs, not by the data.

**(d) The penalty can go negative** when $\rho > \lambda_1/(3\lambda_2)$ — 1.8% of candidates — so merging becomes *rewarded*. With `max_merge` $= \max(n, m)$, unbounded, nothing structurally prevents collapsing a whole sequence into one node. Rare, but unguarded.

**(e) Penalty is coupled to cost statistics.** $\lambda_1, \lambda_2$ summarise the very matrix the penalty is added to, and depend on $w$ — so tuning $w$ silently changes merge behaviour through a second channel, worsening §5's objective.

**(f) Complexity.** $\mathcal{O}(n^2m^2)$ DP states, each doing $\mathcal{O}(n+m)$ work, because `merge_sequence` and `features_correlation` are recomputed in the innermost loop. This is why adversarial mining needs a cluster job.

## 4.1 Remedy

1. **Re-derive the penalty as model selection**: $\beta \cdot (\#\text{units})$, with $\beta$ from MDL — the description length of one segment primitive, following [9] — or simply learned (§5). One scalar, defensible, with a sensitivity curve.
2. **Replace the correlation term with a merge-consistency cost.** The right question is not "are these correlated" but "what do we lose describing this span with one primitive" — the residual of a single trend fit. That is the bottom-up merge criterion of [2], has no magic constant, and is free once $\ell_1$ trend filtering is adopted, since the fit already exists.

3. **Bound `max_merge` to 3–4**: $\mathcal{O}(nmc^2)$ instead of $\mathcal{O}(n^2m^2)$, and justified perceptually — human visual chunking is limited. Report sensitivity to the bound.
4. **Benchmark against the actual competitors.** shapeDTW [10] attaches a shape descriptor per point and sidesteps segmentation entirely, beating DTW on 64 of 84 UCR datasets; SS-DTW does shape-segment DTW. The paper is currently silent on both. If GDTW's edge is interpretability rather than accuracy, state and measure it as such.
5. **Make the merge soft.** Replacing the hard min with a soft-min [11,12] removes the discontinuity and makes the distance differentiable in $\beta$, $\rho$, $w$ — the enabling step for §5.

## 5 Weakness 4: annotators without learning

What exists (`tune_weights.py`) is a 100-trial Optuna search over 12 integer weights, scored by mean Precision@3 across 17 anchors of 9 candidates.

**(a) The objective is a staircase.** Precision@3 with 3 positives takes values in $\{0, \frac{1}{3}, \frac{2}{3}, 1\}$; averaged over 17 anchors it can take at most 52. The recorded run produced **13 distinct values in 100 trials, with 13 tied at the optimum** (Fig. 3). TPE cannot estimate a density on a plateau, so the search is effectively random and the "best" is one arbitrary tie, broken by a max-entropy heuristic unrelated to generalisation.

**(b) The weights are not identifiable.** Condition number $4 \times 10^{12}$, effective rank 7 (§3c): the objective is genuinely flat along roughly five directions. No amount of search fixes this — the 13-way tie is the symptom. Reporting a specific weight vector as a finding about which perceptual features matter is unsupported.

**(c) The reported weights are not the operative ones.** Weights multiply before the $\ell_2$ norm, so $d = \sqrt{\sum_i w_i^2 \delta_i^2}$ and effective importance is $w_i^2$: `mean_value` 0.1665→0.2417 (×1.45), `mean_diff` 0.1560 → 0.2120 (×1.36), `amplitude` 0.0604 → 0.0318 (×0.53), `light_increasing` 0.0113→0.0011 (×0.10), `length` 0.0100→0.0009 (×0.09).

**(d) No held-out evaluation.** Tuning and reporting use the same 17 anchors, so 0.8824 → 0.9412 is a training number with 12 parameters on 17 observations. On that same training set the baselines still win: **DTW 0.9608, LCSS 1.0000, ours 0.9412**. The current evidence does not support the method, and no generalisation claim is possible.

**(e) Annotator information is discarded.** Comparative judgements are reduced to a binary partition, then to a top-$k$ count — throwing away ordering within positives, preference margin, inter-annotator disagreement, and per-annotator reliability.

**(f) Almost nothing is learnable.** Of ≈25 free quantities (6 segmentation constants, the merge constant 3, `max_merge`, 12 standardisation means, 12 s.d.s, 12 weights), only the 12 weights are fitted.

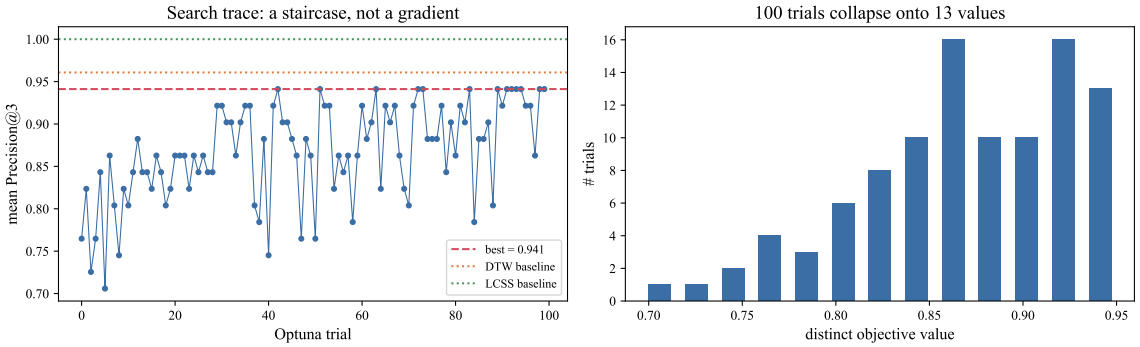


Figure 3: The recorded tuning run. Left: trial trace against the DTW and LCSS baselines. Right: 100 trials collapse onto 13 objective values.

### 5.1 Remedy: make the metric a model

1. **Adopt a comparison likelihood.** For anchor $a$ and candidates $b, c$, model $\Pr(b \text{ preferred}) = \sigma\big((d_\vartheta(a,c) - d_\vartheta(a,b))/\tau\big)$ — the Bradley–Terry / triplet-logistic form [14] — and maximise the log-likelihood. It is continuous, uses every comparison instead of a top-$k$ count, is calibrated, and makes $\tau$ an explicit annotator-noise parameter (per-annotator temperatures or a random effect handle disagreement). This alone removes the plateau of (a).
2. **Make $d_\vartheta$ differentiable and widen $\vartheta$.** With the soft-min DP of §4, gradients reach $\rho$, $\beta$, the weights, and the segmentation $\lambda$: every hand-set constant becomes a fitted parameter. [11] gives the smoothing and its gradient; [12] generalises to arbitrary DPs, which the merge-aware recursion needs.
3. **Replace diagonal weights with a low-rank metric** $d^2 = (x-y)^\top M(x-y)$, $M = A^\top A$, $A \in \mathbb{R}^{r\times 12}$, $r \approx 4$ [17]. This is the correct response to collinearity: learn the directions that matter rather than weighting correlated features independently. Whiten on the segment corpus first and regularise $M$ toward the identity for identifiability.
4. **Fix the protocol.** Grouped $k$-fold CV by anchor; a held-out test set never touched during development; bootstrap CIs over anchors; baselines on the same folds; report effective ($w^2$) importances.

**On annotation size.** 17 anchors cannot support 12 parameters, let alone a metric matrix. Either **collect adaptively** — the adversarial mining already picks informative comparisons, so formalise it as active learning, with acquisition criteria from the crowd-kernel / ordinal-embedding literature [15,16] — or **shrink the model**: fit only $\rho$, $\beta$, $\tau$ by maximum likelihood and derive the rest. Three identifiable parameters fitted honestly is publishable; twelve unidentifiable ones tuned on the training set is not.

**Establish the ceiling.** Fit a free ordinal embedding (t-STE [15], crowd kernel [16]) to the same triplets. It has no interpretability but bounds what *any* model can achieve here and measures annotation consistency. "The interpretable metric attains $x$% of achievable agreement" is a far stronger claim than an unbounded accuracy number, and guards against the case where the annotations are too noisy to separate any two methods.

## 6 Additional defects found during the audit

**Merging on standardised features is algebraically wrong.** `extract_segment_features` returns $z = (\phi - m)/\sigma$; `merge_segments_features` then *sums* the $z$-scores of the additive features (`sum_abs_diff`, `length`, and `amplitude` for same-sign segments). But $z(\phi_1) + z(\phi_2) \neq z(\phi_1 + \phi_2)$ — the gap is a constant $m/\sigma$ per extra merged segment: `length` 1.370 s.d. (**4.11** at a 4-way merge), `sum_abs_diff` 1.021 (3.06), `amplitude` 0.994 (2.98). A merged node is pushed several standard deviations from every unmerged node purely by bookkeeping, regardless of its shape. Since merging *is* the contribution, this is the most consequential defect found. **Fix:** merge in raw space, standardise after. Note weights are also applied *before* the merge (`seq_sim_alg.py:74--75`).

**Standardisation constants are unattributed.** The 12 means and s.d.s at `utils.py:332--337` are literals; no script in the repository computes them. Recompute on the current corpus, check the computation in, and refresh whenever the segmenter changes — which it will, under §2.

**Dead and vestigial code.** Besides §2(d): `mark_nodes_limits` (`utils.py:253`) is never called and contains an `if start == 760: pass` stub; `dtw_merge` has `if i == 4 and j == 4 ...: pass` at `utils.py:413`.

## 7 Prioritised roadmap

Table 4: Recommendations by impact per unit of effort.

| # | § | Action | Effort | Impact |
|---|---|---|---|---|
| 1 | 6 | Merge in raw space, standardise after (removes a multi-s.d. artefact from the core operation) | S | Critical |
| 2 | 5 | Grouped cross-validation + held-out test set; re-report all numbers | S | Critical |
| 3 | 2 | Delete or wire in the dead extremum branch; re-run all results | S | High |
| 4 | 6 | Recompute standardisation constants from the corpus, in a checked-in script | S | High |
| 5 | 3 | Dimensionless features + explicit aspect ratio $\rho$; verify resampling invariance | M | High |
| 6 | 2 | Replace the detector with $\ell_1$ trend filtering or PELT; add the stability metric | M | High |
| 7 | 4 | Merge penalty $\beta$ from reconstruction residual; bound `max_merge` to 3–4 | M | High |
| 8 | 5 | Bradley–Terry likelihood over annotator comparisons | M | High |
| 9 | 5 | Soft-min DP; fit $\rho, \beta, \lambda, w$ by gradient descent | L | High |
| 10 | 3 | Low-rank Mahalanobis metric in place of diagonal weights | L | Medium |
| 11 | 5 | Ordinal-embedding ceiling (t-STE) to bound achievable agreement | M | Medium |
| 12 | 4 | Benchmark against shapeDTW / SSDTW | M | Medium |

Items 1–4 are corrections, not redesigns; do them first, since every current number depends on them and item 1 may change the method's standing against the baselines. Items 5–8 are the redesign and are mutually reinforcing — dimensionless features make the aspect ratio meaningful, a principled segmenter makes the merge residual computable, and a differentiable cost makes all of it learnable from the annotations already being collected. Item 9 unifies them.

## References

[1] F.-l. Chung, T.-c. Fu, R. Luk, V. Ng. "Flexible time series pattern matching based on perceptually important points," *IJCAI Workshop on Learning from Temporal and Spatial Data*, 2001.

[2] E. Keogh, S. Chu, D. Hart, M. Pazzani. "Segmenting time series: a survey and novel approach," in *Data Mining in Time Series Databases*, World Scientific, 2004.
[3] C. Truong, L. Oudre, N. Vayatis. "Selective review of offline change point detection methods," *Signal Processing*, 167:107299, 2020. (`ruptures`.)
[4] R. Killick, P. Fearnhead, I. A. Eckley. "Optimal detection of changepoints with a linear computational cost," *JASA*, 107(500):1590–1598, 2012.
[5] P. Fearnhead, G. Rigaill. "Changepoint detection in the presence of outliers," *JASA*, 114(525):169–183, 2019.
[6] S.-J. Kim, K. Koh, S. Boyd, D. Gorinevsky. "$\ell_1$ trend filtering," *SIAM Review*, 51(2):339–360, 2009.
[7] D. H. Douglas, T. K. Peucker. "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *Cartographica*, 10(2):112–122, 1973.
[8] S. Gharghabi, Y. Ding, C.-C. M. Yeh, K. Kamgar, L. Ulanova, E. Keogh. "Matrix Profile VIII: Domain agnostic online semantic segmentation at superhuman performance levels," *ICDM*, 2017.
[9] T. Rakthanmanon, E. Keogh, S. Lonardi, S. Evans. "MDL-based time series clustering," *Knowledge and Information Systems*, 33(2):371–399, 2012.
[10] J. Zhao, L. Itti. "shapeDTW: Shape Dynamic Time Warping," *Pattern Recognition*, 74:171–184, 2018.
[11] M. Cuturi, M. Blondel. "Soft-DTW: a differentiable loss function for time-series," *ICML*, 2017.
[12] A. Mensch, M. Blondel. "Differentiable dynamic programming for structured prediction and attention," *ICML*, 2018.
[13] G. E. A. P. A. Batista, E. J. Keogh, O. M. Tataw, V. M. A. de Souza. "CID: an efficient complexity-invariant distance for time series," *DMKD*, 28(3):634–669, 2014.
[14] R. A. Bradley, M. E. Terry. "Rank analysis of incomplete block designs: I. The method of paired comparisons," *Biometrika*, 39(3/4):324–345, 1952.
[15] L. van der Maaten, K. Weinberger. "Stochastic triplet embedding," *MLSP*, 2012.
[16] O. Tamuz, C. Liu, S. Belongie, O. Shamir, A. T. Kalai. "Adaptively learning the crowd kernel," *ICML*, 2011.
[17] K. Q. Weinberger, L. K. Saul. "Distance metric learning for large margin nearest neighbor classification," *JMLR*, 10:207–244, 2009.
[18] W. S. Cleveland, M. E. McGill, R. McGill. "The shape parameter of a two-variable graph," *JASA*, 83(402):289–300, 1988.
[19] J. Talbot, J. Gerth, P. Hanrahan. "An empirical model of slope ratio comparisons," *IEEE TVCG*, 18(12):2613–2620, 2012.
[20] A. Gogolou, T. Tsandilas, T. Palpanas, A. Bezerianos. "Comparing similarity perception in time series visualizations," *IEEE TVCG*, 25(1):523–533, 2019.
[21] M. Vlachos, G. Kollios, D. Gunopulos. "Discovering similar multidimensional trajectories," *ICDE*, 2002.
[22] H. Sakoe, S. Chiba. "Dynamic programming algorithm optimization for spoken word recognition," *IEEE TASSP*, 26(1):43–49, 1978.